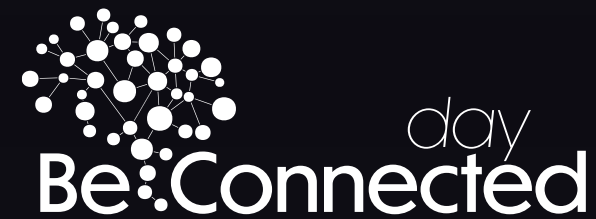


**THE #1 FREE
COMMUNITY
EVENT ON
MICROSOFT
TECHNOLOGY
IN ITALY**



11 JUN

A JOURNEY BEYOND TECHNOLOGY



ShareGate

EPOS



EXHIBO S.p.A.
COMMUNICATION SYSTEMS



SECTION

Sponsors



Gold sponsors



Silver sponsors



Tech partner



Oltre il Low-Code: costruire soluzioni custom integrate con Microsoft 365 nell'era della GenAI

Marco Parenzan

Star Trek addicted

Entra Id tenant collector

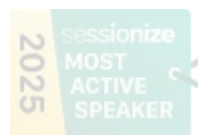


Nice to see you



Marco Parenzan

1nn0va Community Lead



Sono un architetto di soluzioni appassionato e orientato al cliente, con una profonda esperienza in IoT, Cloud, .NET e applicazioni industriali. Microsoft ha riconosciuto i miei contributi con il premio MVP dal 2014 per Azure, e dal 2024 in particolare per Azure IoT.

Con un background come architetto e sviluppatore indipendente, ho costruito una carriera progettando soluzioni scalabili e reali che colmano il divario tra le moderne tecnologie cloud e le esigenze industriali.

Sono un relatore frequente ai principali eventi delle community tecnologiche in tutta Italia, condividendo conoscenze su Azure, IoT e sviluppo .NET.

Come responsabile della community di 1NN0VA, una community tecnologica riconosciuta da Microsoft a Pordenone, Italia, sono appassionata di promuovere l'innovazione, fare da mentore agli sviluppatori e supportare la crescita tecnologica locale.

Nel 2016, ho scritto un libro su Microsoft Azure per aiutare altri ad abbracciare il potenziale della piattaforma.

Nel profondo, sono un ricercatore mancato e un appassionato di retro computing.

Mi piace sviluppare giochi retro e giocare con il suo Commodore 64, immergermi nella fantascienza e nei fumetti, e rilassarmi con una buona serie in streaming.

Agenda



1. Il contesto: perchè si può andare oltre il Low-Code per costruire soluzioni custom integrate con Microsoft 365 nell'era della GenAI
2. Il ruolo della GenAI e di 365
3. Architettura e proposta di soluzione
4. Demo
5. Conclusioni

Il contesto

Una storia

Storia anni '90 (oggi Industrial IoT)

MES in VBA Excel — funzionava bene perché progettato da chi conosceva il processo

- Io ho sviluppato il codice VBA
- Negli anni '90 era abbastanza tipico sviluppare applicazioni “custom”

Un sistema esterno (Delphi) arrivato più tardi non funzionava

- non per colpa dello strumento, ma di chi lo ha fatto
- Chi progetta deve conoscere almeno in parte il processo produttivo ed Dominio
- La qualità non dipende dalla tecnologia: dipende dalla comprensione del dominio

Ad oggi questa pratica è andata perduta: o si trova un applicativo pronto, o spesso viene ancora sostituita da file Excel (dipende ovviamente dalle dimensioni dell'azienda)

Ottimi MES meravigliosi esistono: ma sono costosi e complicati

- Le aziende rinunciano all'implementazione: “costa troppo”

Provocazione: tornare sviluppare applicazioni custom. La GenAI ci può aiutare nell'obiettivo.

Power Apps: filosofia condivisa, limiti reali

Punti di forza	Maschere veloci per il cliente	PowerFX: motore eccellente	Integrazione nativa con 365	Low-code accessibile a non-developer (forse)	Code App con React
Limiti percepiti	Modello applicativo 'a pagine stile app mobile': non adatto a tutti i casi	"Excel è più flessibile"			Code App con React

Non parliamo di costi (anche se sarebbe corretto)

Cosa impariamo dalle Power Apps?

API-based development

L'unico modo per accedere a qualunque risorsa è una REST API
Connettori

Interfaccia utente standard

HTML5 + CSS standard "assodato e conosciuto"

Power BI

Non strettamente Power Apps, ma molto amato da tutti e l'embedding e Fabric aiutano

Con le capacity Fabric, adesso è più «semplice» accedere a scenari «Apps Own Data»

Custom login, no licenze Power BI assegnate a utenti

Embedding report in applicazioni esistenti o in portali custom

Il ruolo della GenAI e di 365

Cit. usare l'AI come chat è come usare un fax

«Se usi l'intelligenza artificiale come una chat — chiedi, lui risponde — stai usando un fax.
Chi non è agentico è già indietro.»

**Personale contributo:
mi trovo benissimo a chattare
almeno così faccio (e giustifico) il mio lavoro
e miglioro comunque le mie prestazioni**

Siamo agli inizi: l'agenticità è potente ma non è l'unico modo di portare valore

- Non è necessario delegare tutto per trarre beneficio dalla GenAI

L'obiettivo non è non lavorare meno, ma lavorare meglio

- La velocità è una conseguenza

La GenAI che coadiuva il nostro lavoro è già un salto di produttività enorme:

- Code review
- Code generator
- Code refactoring

È importante comprendere che la direzione è tracciata:

- Guardarlo con spirito costruttivo è una scelta razionale ed oculata

GenAI: il mito della customizzazione ritorna realtà?

La capacità di Code Generation (il Vibe Coding) permetterebbe di tornare ad uno sviluppo sostenibile di un applicativo custom per un cliente

La GenAI può abbattere diversi costi

- il costo di sviluppo delle UI — il collo di bottiglia storico - HTML5 + CSS3 + JavaScript sono già potentissimi — mancava solo la velocità di sviluppo UI
- API come “Backend for Frontend” a partire da metadati

Per una Azienda, lo sviluppo custom non dovrebbe comunque mai essere “casuale” ma orientate ad un filone di specializzazione

- Approccio di prodotto: ogni commessa fa maturare il prodotto
- Non si reinventa tutto da zero: si generalizza man mano
- Con il code review e il code refactoring si creano e si stabilizzano le porte di personalizzazione

Prompt: importantissimo, ma...

- Il codice stesso diventa una specifica in cui modelli aggiornati stanno diventando efficientissimi

La responsabilità del codice verso il cliente

La GenAI sta mettendo in evidenza la nostra incapacità a dare i requisiti.
Una analogia: come il cloud ha messo in evidenza i costi nascosti.

I prompt sono sostanzialmente una forma di definizione dei requisiti.
Come siamo manchevoli di requisiti nei progetti, così siamo manchevoli nei confronti della GenAI.

Se sono pigro a non controllare riga per riga quello che ha fatto, è una responsabilità mia.

I prompt specificano i requisiti: se i requisiti sono vaghi, il risultato è vago — esattamente come con un junior developer

La capacità esecutiva della GenAI è ordini di grandezza superiore a quella di un junior (e rispetto alla tua)

La direzione la dai tu

La pigrizia nel controllo è una responsabilità del developer, non della GenAI

Qualità dell'implementazione: contratti e interfacce

Definire un contratto, ovvero il corretto (dis)accoppiamento tra produttore e consumatore

- una interfaccia in un linguaggio di programmazione, uno schema di una API, uno modello DB in SQL

Definisci interfacce chiare: la GenAI adatta tutto il codice generato automaticamente

- integra il codice scritto come un requisito alla pari del prompt

La GenAI determina statisticamente l'insieme di token di risposta, quindi è già nella cultura del web

- Se viene specificata una interfaccia, la GenAI adatta tutto il codice a quell'interfaccia (es. Façade della Gang of Four)
- applicato statisticamente: la GenAI “conosce” i principi e li applica (es. La Dependency Injection)

Quindi:

- Il developer definisce: confini, interfacce, responsabilità, requisiti
- La GenAI esegue implementazione, boilerplate, maschere, adattamenti

Capacità esecutiva della GenAI: ordini di grandezza superiore — ma guidata da te

Domanda: non c'è un modo per definire requisiti in maniera diversa/più veloce?

Integrando soluzioni esistenti in modalità SaaS (mutuando l'approccio dal Cloud)

- Più ci si sposta verso destra (SaaS), meno infrastruttura bisogna gestire.
- Cioè si parte da dx: se esiste un SaaS che risolve il problema, la prima domanda non dovrebbe essere "come lo sviluppiamo?", ma "perché dovremmo svilupparlo?"

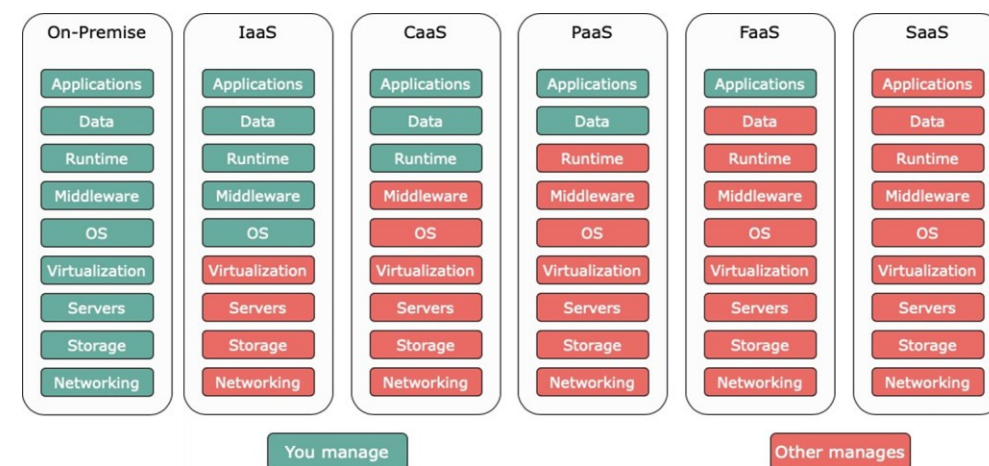
Il Low Code spesso nasce proprio da questa filosofia: "Non scrivere codice se puoi configurare."

La GenAI aggiunge un livello ulteriore:

- Non scrivere manualmente se puoi generare."

La vera produttività non deriva dal produrre software più velocemente, ma dall'evitare di produrlo quando non è necessario.

- Serve davvero costruire un'applicazione o esiste già un servizio che risolve il problema?



Microsoft 365: il pilastro sottovalutato “dai dev” e dal business

Constatazione personale:

- Commesse 365 da una parte
- Sviluppi applicativi dall'altra
- Intersezione reale: quasi solo Entra ID (autenticazione)

365 è un'infrastruttura fatta di API:

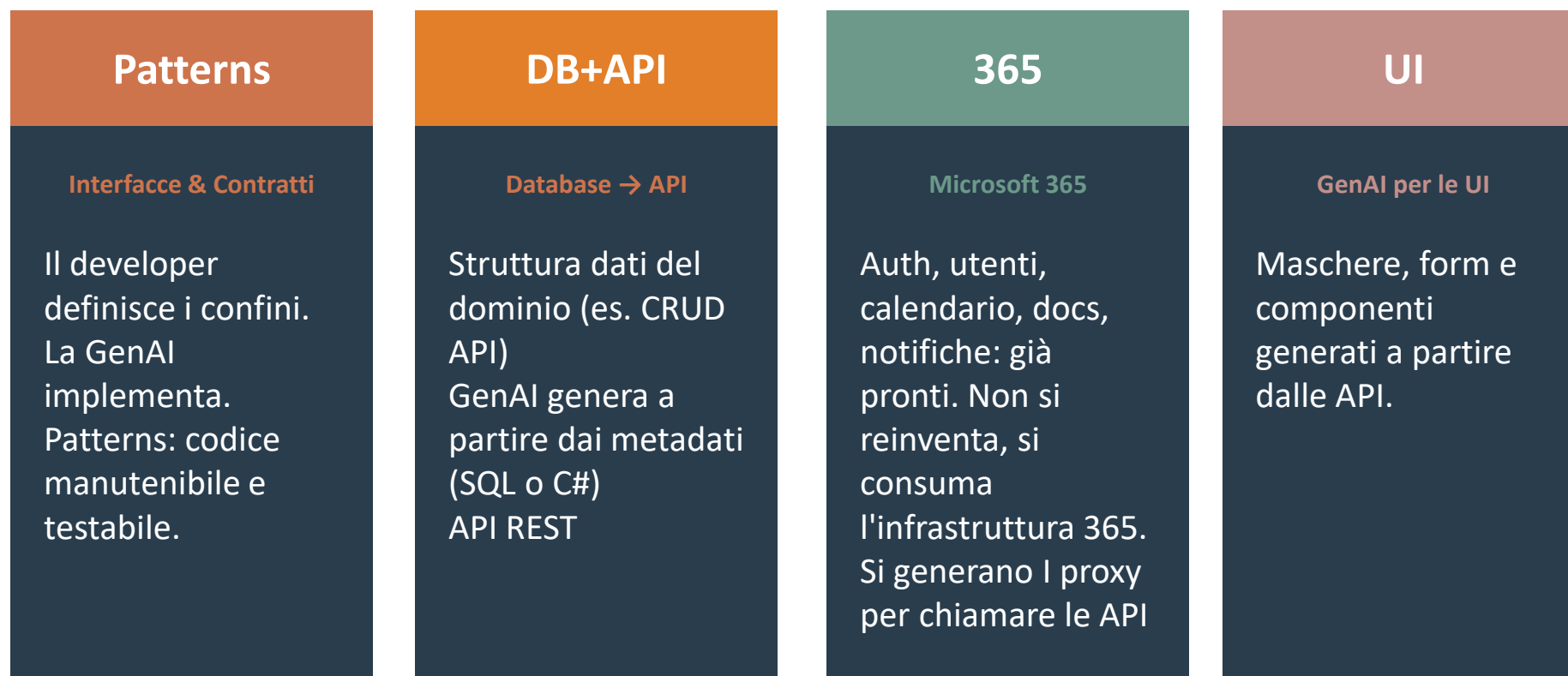
- Autenticazione e gestione utenti (Entra ID)
- Planning e calendario (Calendar API, Planner)
- Archivio documenti (SharePoint, OneDrive)
- Notifiche e messaggistica (Teams, Outlook)

Integrare 365 nativamente è un valore aggiunto concreto — inutile reinventare una funzionalità quando 365 ce la dà già

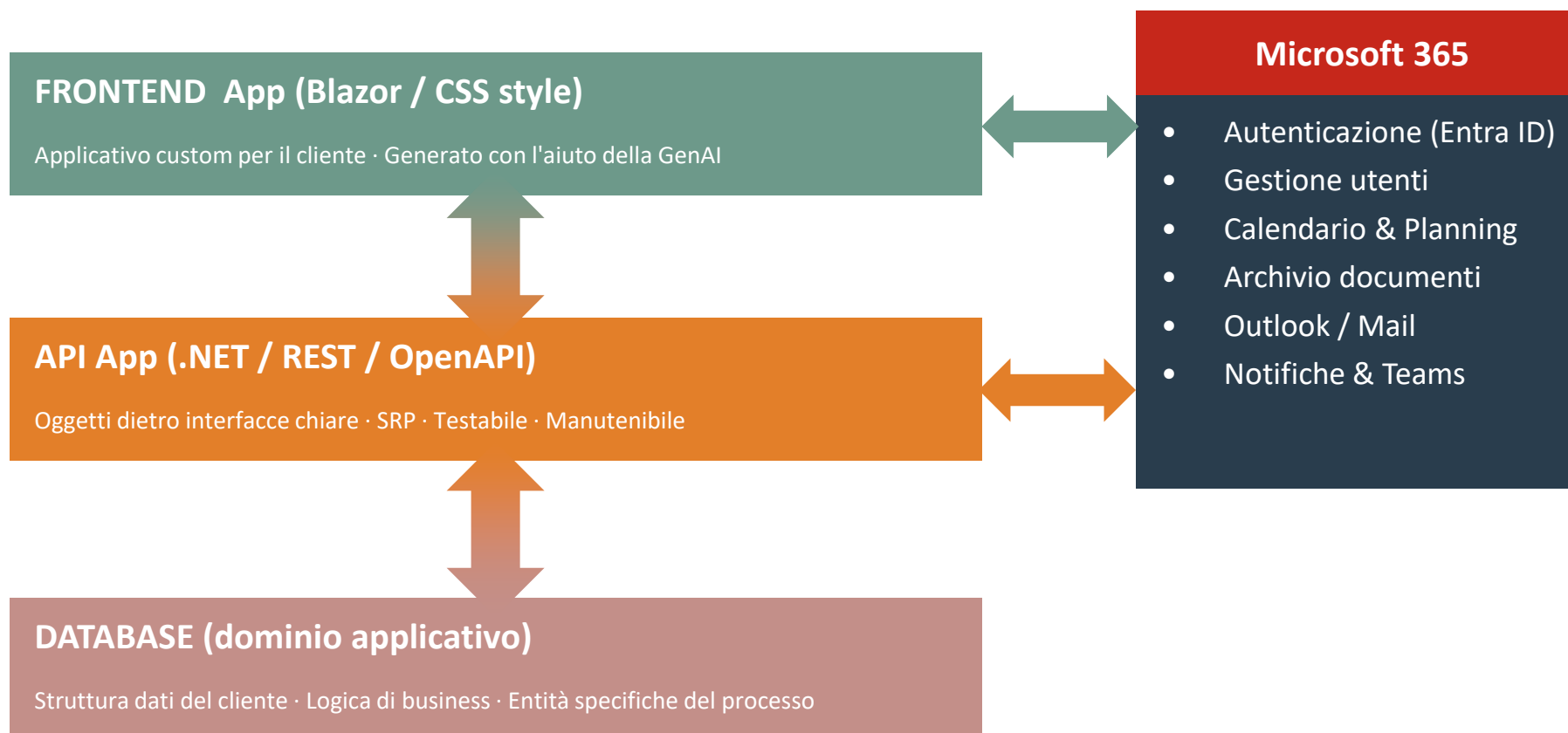
- E fornisce una implementazione robusta che ripaga ampiamente i costi di abbonamento

Architettura e proposta di soluzione

I pilastri della customizzazione sostenibile



L'architettura della soluzione



DEMO

Lo scenario di Demo

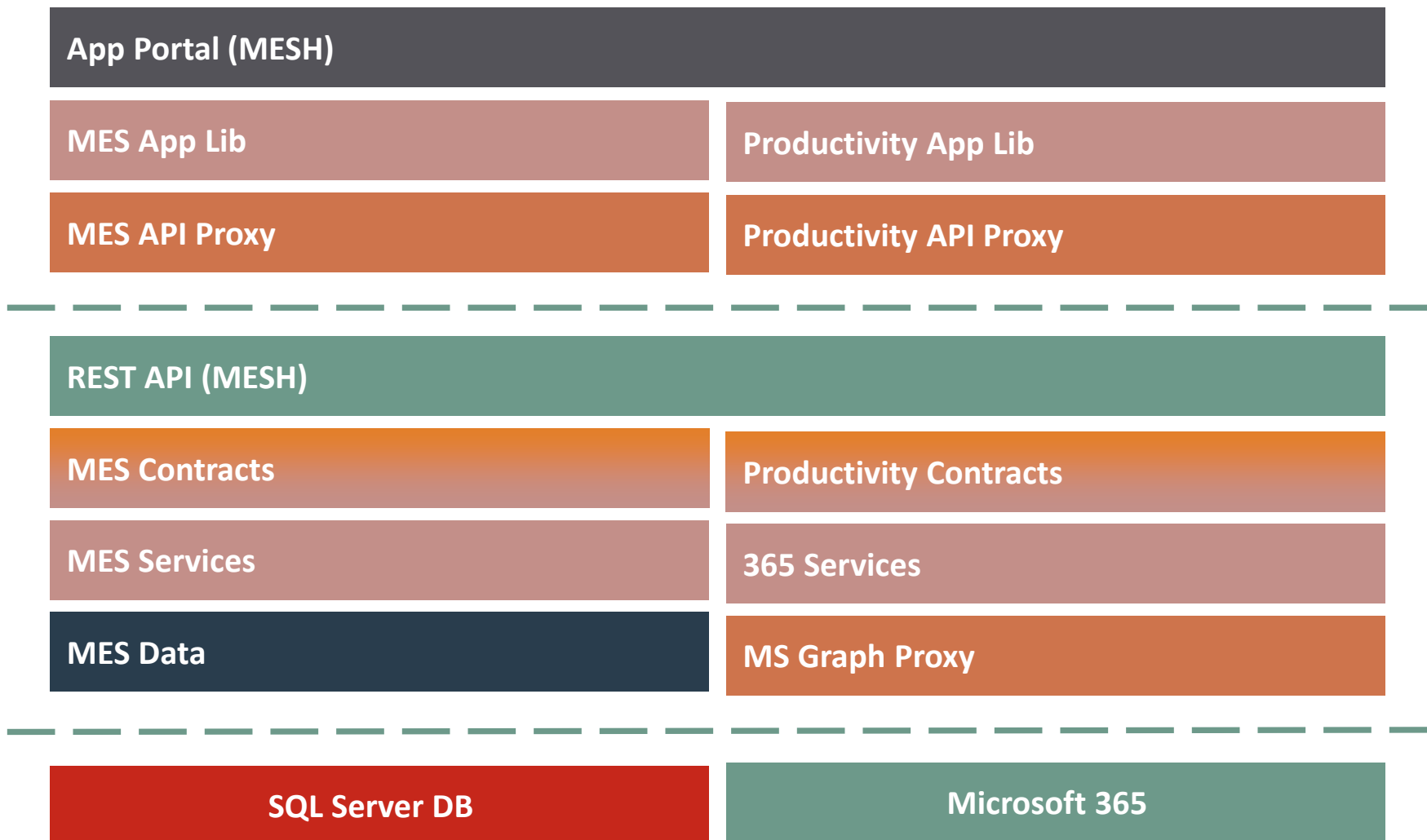
Parliamo di Industrial IoT e di MES

Nell'immaginario di Star Trek, Industrial ➔ Cantiere ➔ Utopia Planitia

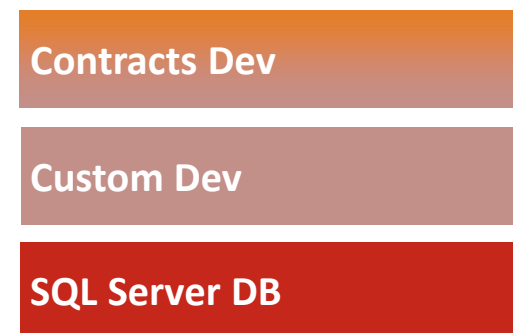
Tenant 365: utopiaforge.onmicrosoft.com

Custom Domain Name: utopiaforge.it

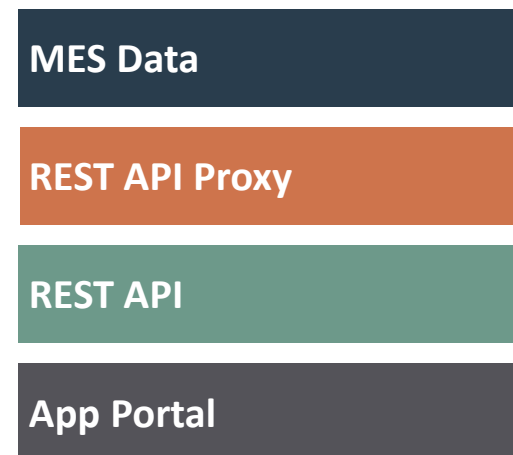
Vertical Slices



CUSTOM



GEN AI



Demo

FASE 01

API Microsoft 365

- Consumare API 365 con GenAI
- Il 'finto Outlook' e il 'finto OneDrive'
- Calendar, Mail, OneDrive API

```
var csvRows = new List<string> { UserId, PrincipalName, DisplayName, TemporaryPassword },  
  
Console.WriteLine();  
Console.WriteLine("Populating Entra ID with");  
Console.WriteLine(" ");  
Console.WriteLine();  
  
var createdCount = 0;  
var skippedCount = 0;
```

```
foreach (var (upn, displayName, givenName, surname) in users)  
{  
    try  
    {  
        var exists = await GetUserAsync(upn, cancellationToken: ct);  
        if (exists != null)  
        {  
            Console.WriteLine($"User {displayName} already exists. Skipping.");  
            skippedCount++;  
            continue;  
        }  
  
        var mail = givenName + surname + "@utopiaforge.it";  
        var password = PasswordGenerator.GeneratePassword(12);  
        var user = await CreateUserAsync(mail, password, cancellationToken: ct);  
        createdCount++;  
    }  
    catch (Exception ex)  
    {  
        Console.WriteLine($"Error creating user {displayName}: {ex.Message}");  
    }  
}
```

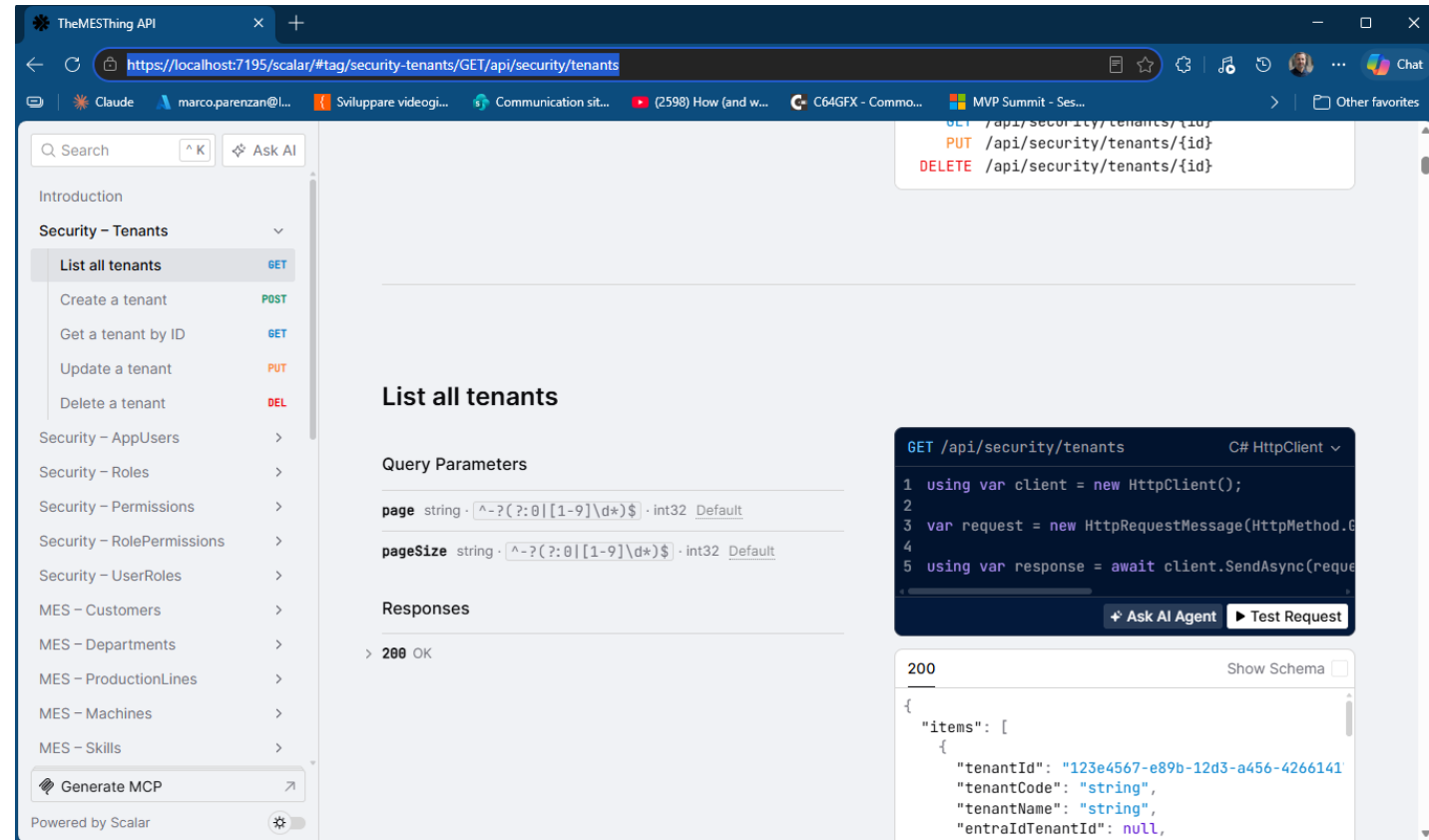
```
2 references  
public async Task<M365User?> GetUserAsync(string userIdOrUpn, CancellationToken ct)  
{  
    try  
    {  
        var user = await _graph.Users[userIdOrUpn].GetAsync(requestOptions =>  
        {  
            req.QueryParameters.Select = UserFields;  
        }, ct);  
  
        return user is null ? null : MapUser(user);  
    }  
    catch (Exception ex)  
    {  
        Console.WriteLine($"Error getting user {userIdOrUpn}: {ex.Message}");  
    }  
}
```

Demo

FASE 02

Database + API

- Dominio applicativo IoT/industrial
- GenAI genera ORM e API CRUD



The screenshot displays the TheMESThing API interface in a web browser. The left sidebar shows a navigation menu with categories like 'Security - Tenants', 'Security - AppUsers', 'Security - Roles', 'Security - Permissions', 'Security - RolePermissions', 'Security - UserRoles', 'MES - Customers', 'MES - Departments', 'MES - ProductionLines', 'MES - Machines', and 'MES - Skills'. The 'List all tenants' endpoint is selected under 'Security - Tenants'. The main panel shows the endpoint details, including the URL 'https://localhost:7195/scalar/#tag/security-tenants/GET/api/security/tenants', the HTTP method 'GET', and the response status '200 OK'. A code editor on the right shows a C# HttpClient request and response example. The response body is a JSON array of tenant objects.

```
GET /api/security/tenants/GET/api/security/tenants
PUT /api/security/tenants/{id}
DELETE /api/security/tenants/{id}
```

List all tenants

Query Parameters

page string · `^~?(?:0|[1-9]\d*)$` · int32 Default

pageSize string · `^~?(?:0|[1-9]\d*)$` · int32 Default

Responses

> 200 OK

```
GET /api/security/tenants C# HttpClient
1 using var client = new HttpClient();
2
3 var request = new HttpRequestMessage(HttpMethod.Get, "/api/security/tenants");
4
5 using var response = await client.SendAsync(request);
6
7 var json = response.Content.ReadAsStringAsync().Result;
8
9 var tenants = JsonSerializer.Deserialize<List<Tenant>>(json);
```

200 Show Schema

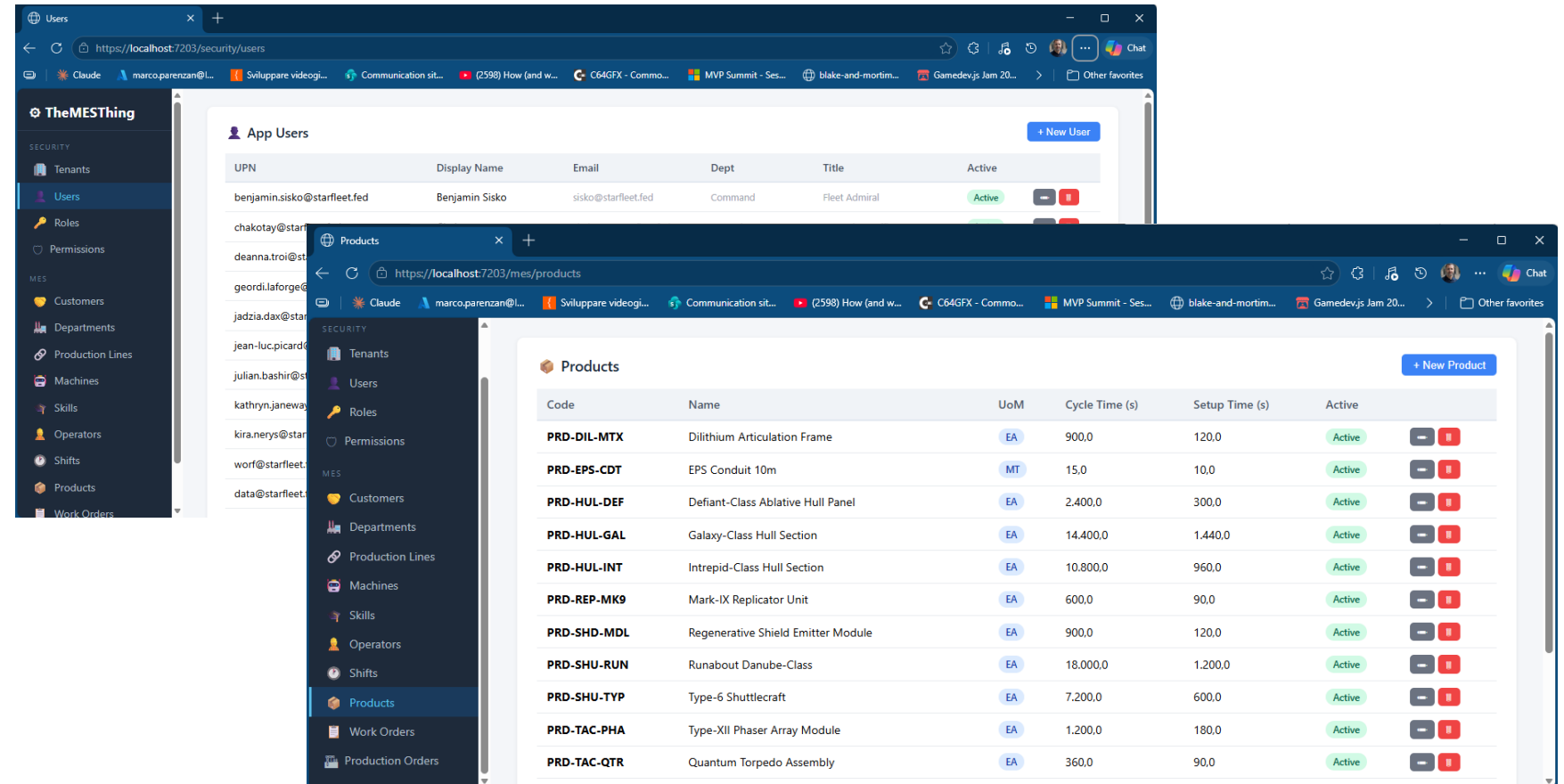
```
{
  "items": [
    {
      "tenantId": "123e4567-e89b-12d3-a456-4266141",
      "tenantCode": "string",
      "tenantName": "string",
      "entraIdTenantId": null,
    }
  ]
}
```

Demo

FASE 03

Composizione finale

- Un solo applicativo coerente
- GenAI genera il proxy REST di frontend
- 365 come infrastruttura di componenti
- Custom dove serve, 365 dove basta



The screenshot displays two overlapping browser windows of the 'TheMESThing' application. The background window shows the 'App Users' management page with a table of users. The foreground window shows the 'Products' management page with a table of products.

App Users Table:

UPN	Display Name	Email	Dept	Title	Active
benjamin.sisko@starfleet.fed	Benjamin Sisko	sisko@starfleet.fed	Command	Fleet Admiral	Active
chakotay@starfleet.fed	Chakotay	chakotay@starfleet.fed	Command	First Officer	Active
deanna.troi@starfleet.fed	Deanna Troi	deanna.troi@starfleet.fed	Medical	Tricorder Medic	Active
geordi.laforge@starfleet.fed	Geordi La Forge	geordi.laforge@starfleet.fed	Engineering	Chief Engineer	Active
jadzia.dax@starfleet.fed	Jadzia Dax	jadzia.dax@starfleet.fed	Science	Science Officer	Active
jean-luc.picard@starfleet.fed	Jean-Luc Picard	jean-luc.picard@starfleet.fed	Command	Captain	Active
julian.bashir@starfleet.fed	Julian Bashir	julian.bashir@starfleet.fed	Medical	Chief Medical Officer	Active
kathryn.janeway@starfleet.fed	Kathryn Janeway	kathryn.janeway@starfleet.fed	Command	Commander	Active
kira.nerys@starfleet.fed	Kira Nerys	kira.nerys@starfleet.fed	Security	Security Officer	Active
worf@starfleet.fed	Worf	worf@starfleet.fed	Security	Security Officer	Active
data@starfleet.fed	Data	data@starfleet.fed	Science	Science Officer	Active

Products Table:

Code	Name	UoM	Cycle Time (s)	Setup Time (s)	Active
PRD-DIL-MTX	Dilithium Articulation Frame	EA	900.0	120.0	Active
PRD-EPS-CDT	EPS Conduit 10m	MT	15.0	10.0	Active
PRD-HUL-DEF	Defiant-Class Ablative Hull Panel	EA	2,400.0	300.0	Active
PRD-HUL-GAL	Galaxy-Class Hull Section	EA	14,400.0	1,440.0	Active
PRD-HUL-INT	Intrepid-Class Hull Section	EA	10,800.0	960.0	Active
PRD-REP-MK9	Mark-IX Replicator Unit	EA	600.0	90.0	Active
PRD-SHD-MDL	Regenerative Shield Emitter Module	EA	900.0	120.0	Active
PRD-SHU-RUN	Runabout Danube-Class	EA	18,000.0	1,200.0	Active
PRD-SHU-TYP	Type-6 Shuttlecraft	EA	7,200.0	600.0	Active
PRD-TAC-PHA	Type-XII Phaser Array Module	EA	1,200.0	180.0	Active
PRD-TAC-QTR	Quantum Torpedo Assembly	EA	360.0	90.0	Active

Conclusioni

Conclusioni



Il tempo effettivo dedicato alla preparazione della demo, tutto quello che troverete nel repo, sicuramente molto meno di 16 ore, forse circa 8-10.

Il tempo di ragionamento e pianificazione, dovuto anche ad altri progetti, tante più ore o meglio giorni.

La direzione è tracciata

Volenti o nolenti, la GenAI è parte del futuro dello sviluppo software. Guardarlo con spirito costruttivo è la scelta razionale.

La responsabilità rimane tua

La GenAI esegue. Tu definisci: confini, interfacce, requisiti, qualità. La pigrizia nel controllo è responsabilità del developer.

365 + .NET + GenAI = opportunità

Integrare 365 come infrastruttura, sviluppare custom con GenAI, applicare SRP: un trionfo concreto e praticabile oggi.

Customizzazione sostenibile

Le commesse maturano in prodotto. L'investimento si ammortizza. Non è un one-shot: è un approccio industriale alla personalizzazione.

Q&A



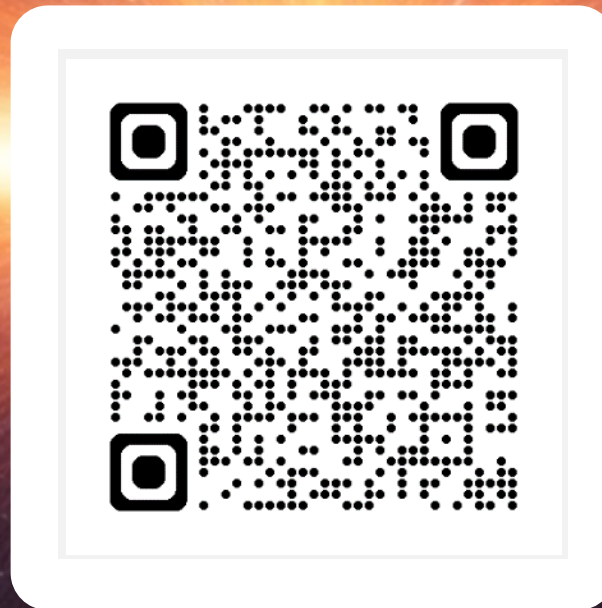
marcoparenzan



<https://github.com/marcoparenzan/TheMESThing>

We care about your feedback

Please take a moment to tell us your opinion using the
run.events app.





GRAZIE

A STAR TREK FAN PRODUCTION. Star Trek and all related marks, logos and characters are solely owned by CBS Studios Inc. This fan production is not endorsed by, sponsored by, nor affiliated with CBS, Paramount Pictures, or any other Star Trek franchise, and is a non-commercial fan-made film intended for recreational use. No commercial exhibition or distribution is permitted. No alleged independent rights will be asserted against CBS or Paramount Pictures.